

Smart Contract Security Analysis Report

Generated on: 2026-04-20 23:13:15

1. Executive Summary

This report provides a consolidated view of the static analysis performed on 10 smart contracts. In total, 6 potential vulnerabilities were identified across 6 contracts.

2. Findings Overview

Contract Name	Total Issues	Status
AccessControl.sol	1	VULNERABLE
Reentrancy.sol	1	VULNERABLE
SelfDestruct.sol	1	VULNERABLE
TxOrigin.sol	1	VULNERABLE
UncheckedCalls.sol	0	SECURE
contract_2.sol	0	SECURE
contract_3.sol	0	SECURE
contract_4.sol	1	VULNERABLE
contract_5.sol	0	SECURE
contract_7.sol	1	VULNERABLE

3. Detailed Vulnerability Analyses

Contract: AccessControl.sol

Issue: Missing Access Control (Severity: Critical)

Function: `changeOwner` | Line: 13

Explanation:

The function 'changeOwner' updates the sensitive variable 'owner' but does not seem to have any access control

Smart Contract Security Analysis Report

Generated on: 2026-04-20 23:13:15

(like onlyOwner). This allows any external actor to take control of the contract.

Suggest

Add an access control modifier (e.g., 'onlyOwner') to the function 'changeOwner' at line 13.

Contract: contract_4.sol

Issue: Reentrancy (CEI Violation) (Severity: High)

Function: withdraw | Line: 11

Explanation:

The external call at line 11 occurs before state variables (balances) are updated at line 13. This violates the Checks-Effects-Interactions pattern.

Suggest

Reorder the operations in function 'withdraw' to update the state variables (balances) before the external call at line 11.

Contract: contract_7.sol

Issue: Reentrancy (CEI Violation) (Severity: High)

Function: withdraw | Line: 10

Explanation:

The external call at line 10 occurs before state variables (balances) are updated at line 13. This violates the Checks-Effects-Interactions pattern.

Suggest

Reorder the operations in function 'withdraw' to update the state variables (balances) before the external call at line 10.

Contract: Reentrancy.sol

Issue: Reentrancy (CEI Violation) (Severity: High)

Function: withdraw | Line: 16

Explanation:

The external call at line 16 occurs before state variables (balances) are updated at line 19. This violates the Checks-Effects-Interactions pattern.

Suggest

Reorder the operations in function 'withdraw' to update the state variables (balances) before the external call at line 16.

Smart Contract Security Analysis Report

Generated on: 2026-04-20 23:13:15

Contract: SelfDestruct.sol

Issue: Unprotected Self-Destruct (Severity: Critical)

Function: kill | Line: 13

Explanation:

The function 'kill' contains a self-destruct operation at line 13 and lacks proper access control. Any external user can call this to destroy the contract.

Restrict access to the function 'kill' using an 'onlyOwner' modifier or remove the self-destruct call.

Suggest

Contract: TxOrigin.sol

Issue: Vulnerable use of tx.origin (Severity: High)

Function: withdrawAll | Line: 13

Explanation:

The function 'withdrawAll' uses 'tx.origin' for authorization at line 13. This makes the contract vulnerable to phishing attacks where an attacker can trick a user into calling a malicious contract that then calls this function.

Replace 'tx.origin' with 'msg.sender' in function 'withdrawAll' to ensure only the immediate caller is authenticated.

Suggest